Deep Learning - COSC2779

Neural Network Model Interpretation

A/Prof. Ruwan Tennakoon

RMIT UNIVERSITY

Outline

1. Feature Visualization
2. Feature Attribution
   - Model-agnostic: LIME and SHAP
3. Concepts
4. Interpretable by Design: Concept Bottlenecks
5. Explaining Sequence Models
   - Token Attribution Basics
   - Explaining Transformers
6. Interpreting Vision-Language Models

Deep Learning Models

- So far we explored how to develop models for image, text, voice, . . . , analysis.
- Usually involve:
  1. Determine your goals
  2. Default Baseline Model: Baseline Model architecture, cost functions, optimization.
  3. Setup the diagnostic instrumentation
  4. Make incremental changes
- Performance of the model is evaluated on an **independent and identically distributed** test set.

Deep Learning Models

- So far we explored how to develop models for image, text, voice, . . . , analysis.
- Usually involve:
  1. Determine your goals
  2. Default Baseline Model: Baseline Model architecture, cost functions, optimization.
  3. Setup the diagnostic instrumentation
  4. Make incremental changes
- Performance of the model is evaluated on an **independent and identically distributed** test set.

- Many see Deep Networks an **black boxes**.
- Important questions that need to be answered are:
  - **How can I trust your model?**
  - **How does your model really make its decisions?**
- Performance on iid test set alone cannot answer these questions.

How does your model really make its decisions?

Urban Legend: "Once upon a time, the US Army wanted to use neural networks to automatically detect camouflaged enemy tanks. The researchers trained a neural net on 50 photos of camouflaged tanks in trees, and 50 photos of trees without tanks. Using standard techniques for supervised learning... Without further training the neural network classified all remaining (test) photos correctly. It turned out that in the researchers' dataset, photos of camouflaged tanks had been taken on cloudy days, while photos of plain forest had been taken on sunny days. The neural network had learned to distinguish cloudy days from sunny days, instead of distinguishing camouflaged tanks from empty forest."



How does your model really make its decisions?

Urban Legend: "Once upon a time, the US Army wanted to use neural networks to automatically detect camouflaged enemy tanks. The researchers trained a neural net on 50 photos of camouflaged tanks in trees, and 50 photos of trees without tanks. Using standard techniques for supervised learning... Without further training the neural network classified all remaining (test) photos correctly. It turned out that in the researchers' dataset, photos of camouflaged tanks had been taken on cloudy days, while photos of plain forest had been taken on sunny days. The neural network had learned to distinguish cloudy days from sunny days, instead of distinguishing camouflaged tanks from empty forest."

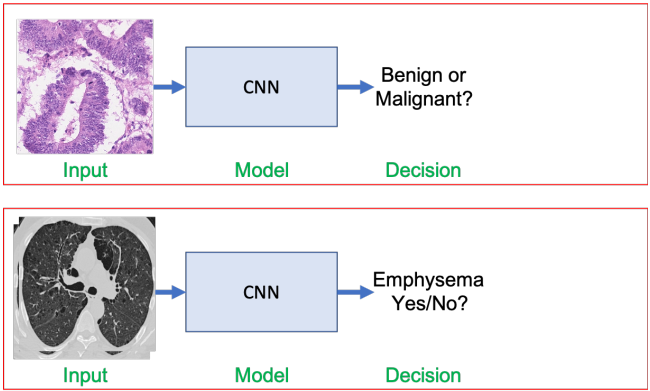


What we expected: A neural network which detects camouflaged army tanks.

What we got: A neural network which distinguishes between cloudy and sunny days

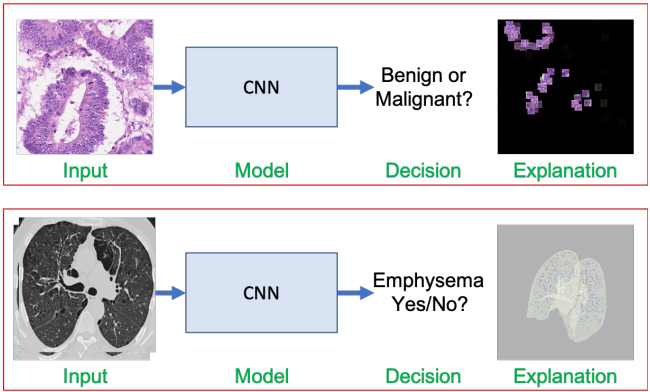
Good story. It's very likely, though not certain, that this didn't actually happen.

Why did the model make a particulate decision?



Why does the model say input is malignent (or has emphysema)?

Why did the model make a particulate decision?



Tennakoon, R., et. al. "Classification of Volumetric Images Using Multi-Instance Learning and Extreme Value Theorem".IEEE Transactions on Medical Imaging, 2019.

If you can ensure that the machine learning model can explain decisions, you can also check the following traits more easily (Doshi-Velez and Kim 2017):

- **Trust**: It is easier for humans to trust a system that explains its decisions compared to a black box.
- **Causality**: Check that only causal relationships are picked up.
- **Reliability or Robustness**: Ensuring that small changes in the input do not lead to large changes in the prediction.
- **Fairness**: Ensuring that predictions are unbiased and do not implicitly or explicitly discriminate against protected groups. An interpretable model can tell you why it has decided that a certain person should not get a loan, and it becomes easier for a human to judge whether the decision is based on a learned demographic (e.g.racial, gender) bias.
- **Privacy**: Ensuring that sensitive information in the data is protected.

Doshi-Velez, F. and Kim, B., 2017. Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608

Methods and models that make the behaviour and predictions of **machine learning** systems understandable to humans.

- Intrinsic or post hoc.
  - **Intrinsic**: Using a machine learning model which are interpretable in nature (linear models or tree-based models).
  - **Post hoc**: Using a black box model (ensemble methods or neural networks) and applying interpretability methods after.

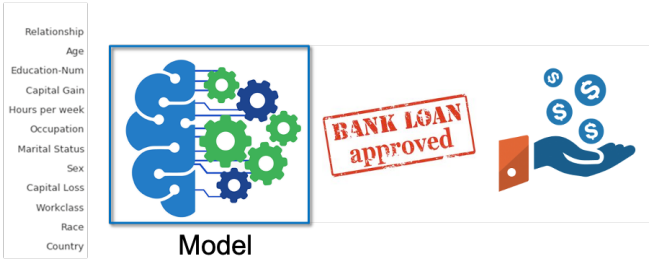
Methods and models that make the behaviour and predictions of **machine learning** systems understandable to humans.

- Intrinsic or post hoc.
  - **Intrinsic**: Using a machine learning model which are interpretable in nature (linear models or tree-based models).
  - **Post hoc**: Using a black box model (ensemble methods or neural networks) and applying interpretability methods after.
- Model-specific or model-agnostic
  - **Model-specific**: specific to intrinsic model interpretation methods which depend purely on the capabilities and features on a per-model basis.
  - **Model-agnostic**: Mostly post hoc methods that can be used on any machine learning model. These methods usually operate on feature input and output pairs.

Methods and models that make the behaviour and predictions of **machine learning** systems understandable to humans.

- Intrinsic or post hoc.
  - **Intrinsic**: Using a machine learning model which are interpretable in nature (linear models or tree-based models).
  - **Post hoc**: Using a black box model (ensemble methods or neural networks) and applying interpretability methods after.
- Model-specific or model-agnostic
  - **Model-specific**: specific to intrinsic model interpretation methods which depend purely on the capabilities and features on a per-model basis.
  - **Model-agnostic**: Mostly post hoc methods that can be used on any machine learning model. These methods usually operate on feature input and output pairs.
- Local or global
  - **Local**: Explains a single prediction
  - **Global**: Explains the behaviour of the the entire model

Local Vs Global



- **Local**: Why did (not) my loan get approved?
- **Global**: Is The model biased to a particular race or gender?

- **Feature Visualization**: What features has the neural network learned?
- **Feature Attribution** : How did each input contribute to a particular prediction?
- **Concepts**: Which more abstract concepts has the neural network learned?
- **Model Distillation**: How can we explain a neural network with a simpler model?

Outline

1. Feature Visualization
2. Feature Attribution
   - Model-agnostic: LIME and SHAP
3. Concepts
4. Interpretable by Design: Concept Bottlenecks
5. Explaining Sequence Models
   - Token Attribution Basics
   - Explaining Transformers
6. Interpreting Vision-Language Models

What are the intermediate layers in a neural network looking for?



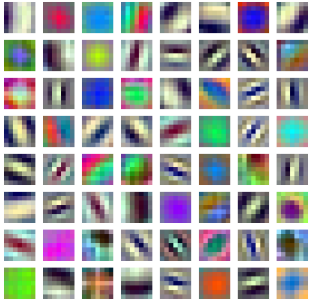


German Shepherd

Image of the dog: Unsplash

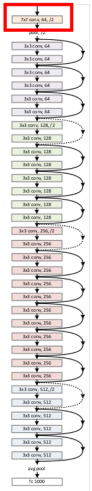
Global Interpretability: Explains what the network has learned?

First Layer: Visualize Filters

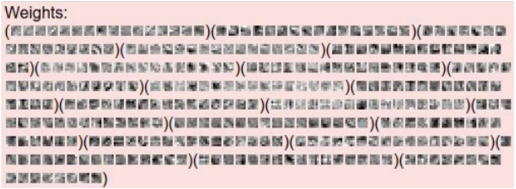


ImageNet trained weights for ResNet50V2.

Plot each weight filter of the first convolution layer. ResNet50 First Conv layer 64 filters, each 7x7 kernel with 3 channels.
Similar patterns seen in many CNNs.

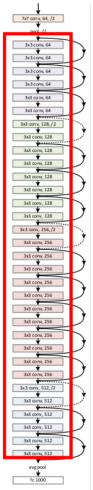


Middle Layers: Visualize Filters

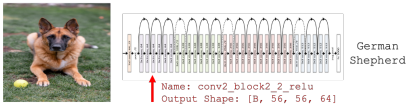


From ConvNetJS CIFAR-10 demo

We can visualize filters in middle layers, but not that interesting.



Middle Layers: Visualize Outputs

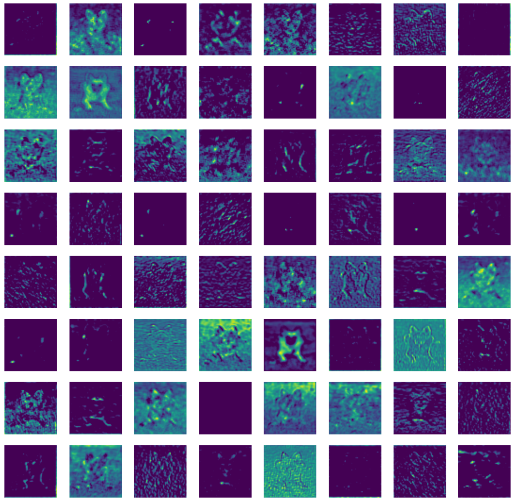


From ConvNetJS CIFAR-10 demo

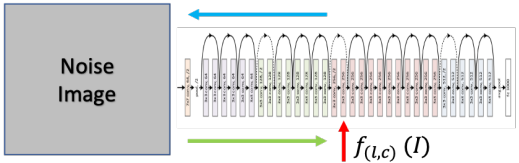
We can visualize Outputs of middle layers.

- Forward propagate for the image of interest.
- Extract the output at any layer and plot each channel separately.

Observing a particular filter for many inputs will provide an insight into what it has learned.



Generate a synthetic image that maximally activated a given neuron.

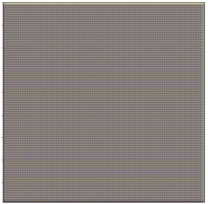


$$I_{syn} = \arg\max_{I} f_{(l,c)}(I) + R(I)$$

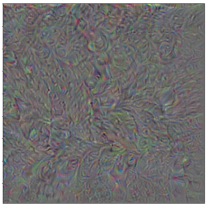
- $f_{(l,c)}(I)$: Activation of layer l channel c for input image I.
- $R(I)$: Some regularization function on image I.

- Initialize image to random noise.
- Repeat
  1. Forward image to calculate current score.
  2. Backprop to get gradient of score with respect to image pixels.
  3. Update the image pixel values based on gradients.

Visualizing Features: Gradient Ascent

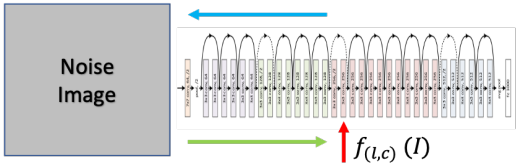


Layer 1: ResNet50



Layer 5: ResNet50

From ResNet50 trained on ImageNet.



- Initialize image to random noise.
- Repeat
  1. Forward image to calculate current score.
  2. Backprop to get gradient of score with respect to image pixels.
  3. Update the image pixel values based on gradients.

Visualizing Features: Gradient Ascent

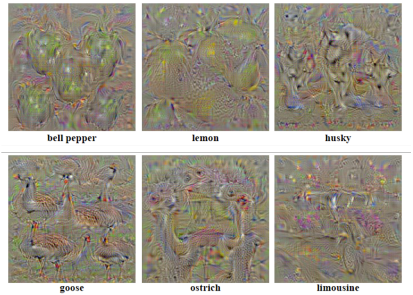
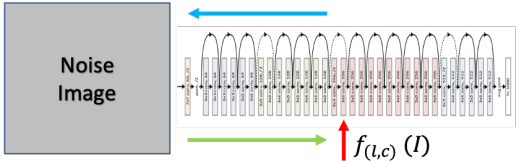


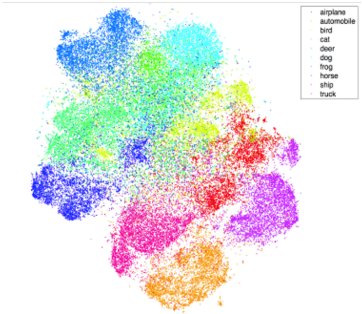
Image: Simonyan, K., Vedaldi, A. and Zisserman, A., 2013. Deep inside convolutional networks: Visualising image classification models and saliency maps.

Better regularization lead to better visualizations.



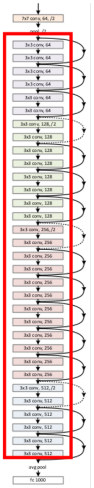
- Initialize image to random noise.
- Repeat
  1. Forward image to calculate current score.
  2. Backprop to get gradient of score with respect to image pixels.
  3. Update the image pixel values based on gradients.

Fully Connected Layers: Visualize Filters



tSNE visualization of CIFAR10 data of final fully connected layer.

We can visualize filters in middle layers, but not that interesting



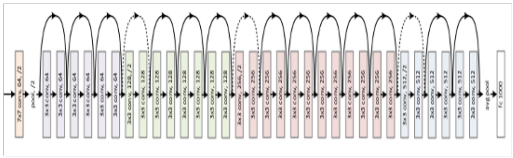
- Pros:
  - Feature visualizations give unique insight into the working of neural networks.
  - Through feature visualization, we have learned that neural networks first learn simple edge and texture detectors and more abstract part and object detectors in higher layers.
  - Communicate in a non-technical way how neural networks work.
- Cons:
  - Many feature visualization images are not interpretable at all.
  - There are too many units to look at.
  - Illusion of Interpretability?

Outline

1. Feature Visualization
2. Feature Attribution
   - Model-agnostic: LIME and SHAP
3. Concepts
4. Interpretable by Design: Concept Bottlenecks
5. Explaining Sequence Models
   - Token Attribution Basics
   - Explaining Transformers
6. Interpreting Vision-Language Models

How did each input contribute to a particular prediction?



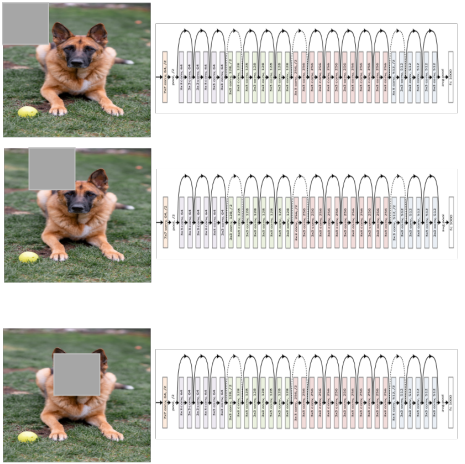


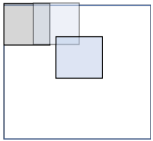
German Shepherd

Image of the dog: Unsplash

Local interpretability: Explains the rational for single input.

Which pixels matter: Mask part of the image before it is fed to the CNN. See how much the prediction probability for a class change.

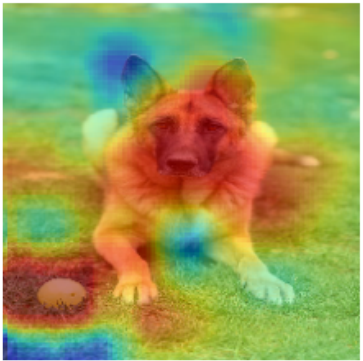




Which pixels matter: Mask part of the image before it is fed to the CNN. See how much the prediction probability for a class change.

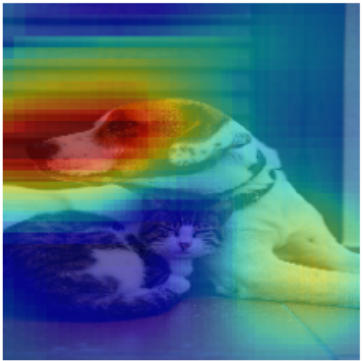


Class: German Shepard

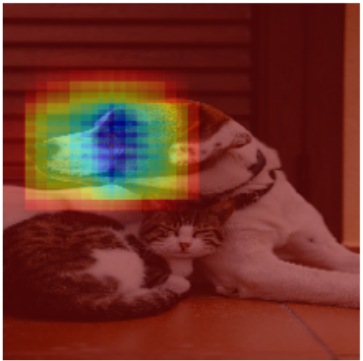


Class: Tennis Ball

Which pixels matter: Mask part of the image before it is fed to the CNN. See how much the prediction probability for a class change.

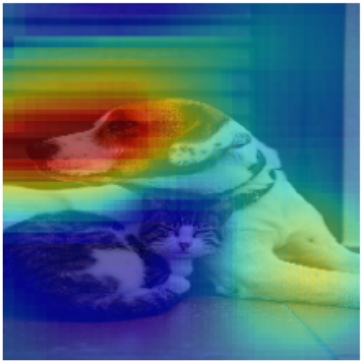


Class: Walker hound

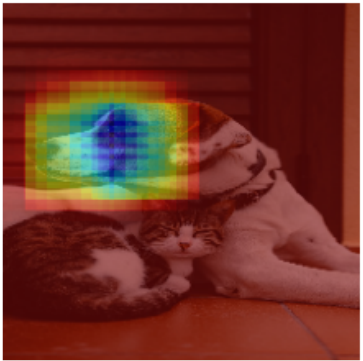


Class: Tabby Cat

Which pixels matter: Mask part of the image before it is fed to the CNN. See how much the prediction probability for a class change.

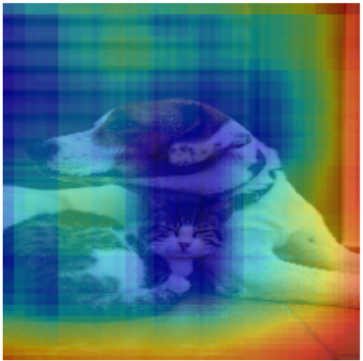


Class: Walker hound

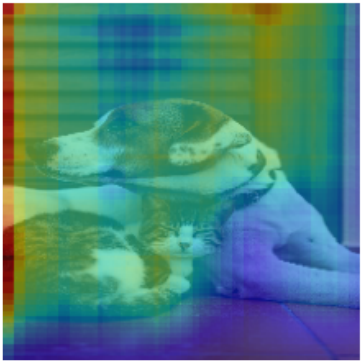


Class: Tabby Cat

What if the network is not correctly trained: Lets check Saliancy map for randomly initialized weights.



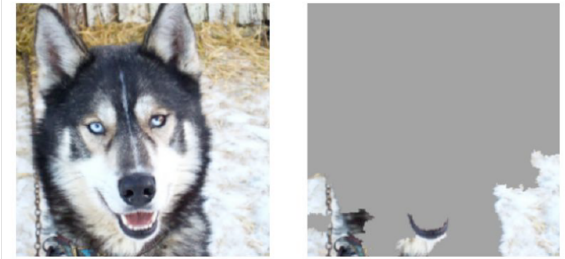
Class: Walker hound



Class: Tabby Cat

There are better ways to occlude images than rectangle patches.

Such methods can also find biases and explain why the model is making mistakes.



(a) Husky classified as wolf (b) Explanation

Image: Ribeiro et al. "Why should I trust you?" Explain the predictions of any classifier. 2016.

ResNet50 trained on imagenet does not make such mistakes as it pays attention to face.

Which pixels matter: Compute gradient of class score with respect to image pixels. Take absolute value and max over RGB channels.

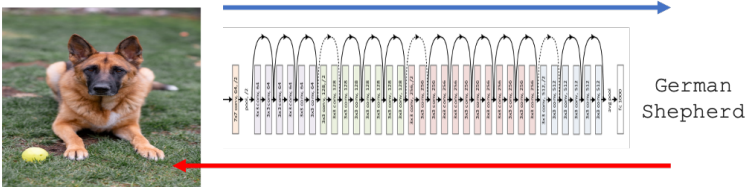
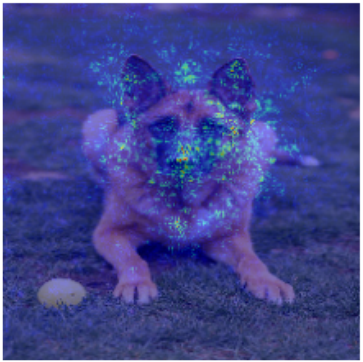
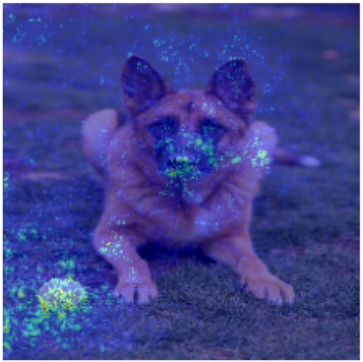


Image of the dog: Unsplash

Which pixels matter: Compute gradient of class score with respect to image pixels. Take absolute value and max over RGB channels.



Class: German Shepard



Class: Tennis Ball

Which pixels matter: Compute gradient of class score with respect to image pixels. Take absolute value and max over RGB channels.

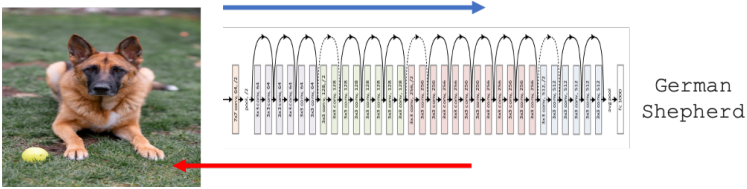


Image of the dog: Unsplash

This technique can also be used to check what pixels in an image maximally activate a given neuron.

Modern CNN architectures (e.g. ResNet, GoogleNet) has the following general structure:

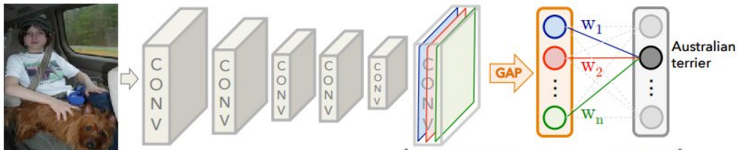


Image: Zhou et. al. CVPR 2016

- The feature elements going into the last linear classifier can be thought of as an 'attribute' in the image. E.g. element 1 maybe for human faces, element 2 for dog faces, ...
- The 'attributes' location information should be in the activation going into the GAP.
- In linear classifier feature importance is related to weights.

Modern CNN architectures (e.g. ResNet, GoogleNet) has the following general structure:

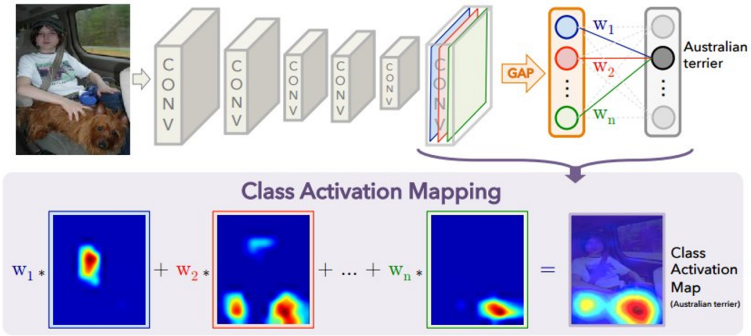


Image: Zhou et. al. CVPR 2016

An Improvement that combines gradient based saliency and Class Activation Maps:

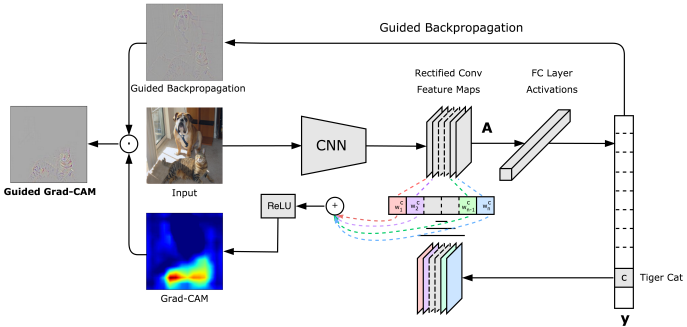


Image: Salvaraju, R et. al. ICCV 2017

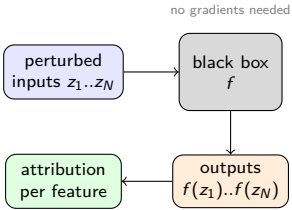
Model-Agnostic Attribution

Saliency, CAM and Grad-CAM need to look *inside* the network (gradients, feature maps, a specific architecture).

Model-agnostic methods treat the model as a **black box**: *perturb* the input, query f, and watch how the output changes. They work for *any* model – even one behind an API.

Occlusion was a first example. Two principled, widely used methods:

- **LIME**: fit a simple model that mimics f *near* x.
- **SHAP**: share the prediction out fairly among the features (game theory).



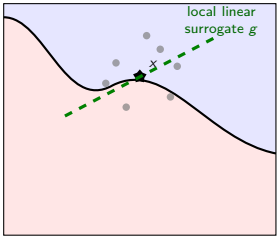
LIME: Local Interpretable Model-agnostic Explanations

Ribeiro et al., KDD 2016

Key idea: a complex decision boundary is non-linear *globally* but **approximately linear locally** – so explain one prediction with a simple model valid *around that input*.
LIME finds a simple surrogate g (e.g. sparse linear):

$$\xi(x) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

- $\mathcal{L}$: how badly g mimics f on perturbed samples, weighted by closeness π_x;
- $\Omega(g)$: complexity penalty (e.g. at most K features);
- the weights of g are the explanation.

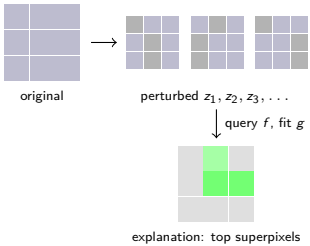


Complex boundary (solid); samples near x (grey); the dashed line fits f only locally.

LIME for Images

Pixels are poor features for a human, so LIME uses **superpixels** (regions of similar colour):

1. Segment the image into M superpixels.
2. Make N perturbed images by randomly **greying out** superpixels; encode each as $z \in \{0,1\}^M$ ($1 =$ kept).
3. Query the black box: $f(z_1), \ldots, f(z_N)$.
4. Weight each sample by its similarity to the original.
5. Fit a weighted sparse linear model $g(z) = w_0 + \sum_j w_j z_j$.
6. Show the superpixels with the largest positive w_j.



Strengths

- Any model, any data type (images, text, tabular).
- Explanation in human units: superpixels, words, feature values.
- Sparse – shows only the few features that matter most.

Pitfalls

- **Instability**: random sampling can give different explanations on different runs.
- **Kernel width** (what counts as "local") changes the result, and there is no principled choice.
- **Out-of-distribution samples**: greyed-out images are unlike training data, so f's behaviour on them may be unreliable.
- Depends on the **segmentation** – change the superpixels, change the story.
- Linear surrogate may be a poor local fit.

Treat prediction as a **cooperative game**:

- **players** = features (pixels, superpixels, words);
- **payout** = the model's prediction;
- question: how to **fairly** divide the payout among the players?

The Shapley value of feature i is its **average marginal contribution** over all orders in which features could be added:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\ (|F|-|S|-1)!}{|F|!} \Big[v(S \cup \{i\}) - v(S) \Big]$$

- F: all features; S: a coalition that does not contain i;
- $v(S)$: the prediction when only features in S are "present" (the others replaced by a baseline).

Intuition

Add features one at a time in a random order. Each time feature i joins, record how much the prediction jumps. Average that jump over *all* possible orders.

The weight $\frac{|S|!(|F|-|S|-1)!}{|F|!}$ is just the fraction of orderings in which exactly the coalition S comes before i.

SHAP: A Worked Example

A dog classifier, image split into three regions: **H**ead, **B**ody, back**G**round. $v(S) = p(\text{dog})$ with only regions in S visible:

coalition S	$v(S)$
$\varnothing$ (baseline)	0.2
{H}, {B}, {G}	0.5, 0.3, 0.2
{H,B}, {H,G}, {B,G}	0.8, 0.6, 0.4
{H,B,G} (full image)	0.9

Marginal contribution of each region in each of the $3! = 6$ orders:

order	H	B	G
H, B, G	0.3	0.3	0.1
H, G, B	0.3	0.3	0.1
B, H, G	0.5	0.1	0.1
B, G, H	0.5	0.1	0.1
G, H, B	0.4	0.3	0.0
G, B, H	0.5	0.2	0.0
average ϕ	**0.42**	**0.22**	**0.07**

Check: $0.42 + 0.22 + 0.07 \approx 0.7 = 0.9 - 0.2$. The head explains most of the "dog" score; the background almost nothing.

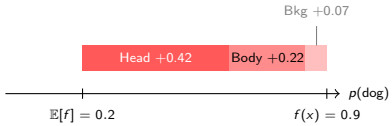
Why Shapley Values? The Fairness Properties

Shapley values are the **unique** attribution satisfying:

- **Efficiency** (local accuracy): the attributions add up exactly to the prediction minus the baseline,

$$\sum_i \phi_i = f(x) - \mathbb{E}[f(X)].$$

- **Symmetry**: two features that always contribute equally get equal credit.
- **Dummy** (missingness): a feature that never changes the output gets $\phi_i = 0$.
- **Additivity**: for a sum of models, the attributions add.



A "force plot": the features push the prediction from the average output to this image's output.

Computing SHAP in Practice

Exact values need $v(S)$ for all 2^M coalitions – infeasible for many features.

Approximations:

- **KernelSHAP**: LIME with a special weighting – fit a weighted linear model on sampled coalitions using the *Shapley kernel*

$$\pi(z) = \frac{M-1}{\binom{M}{|z|}\,|z|\,(M-|z|)}$$

  Its coefficients approximate Shapley values.
- **DeepSHAP / GradientSHAP**: use gradients – fast for deep nets.
- **TreeSHAP**: exact and fast for trees.

Practical choices

- For images, use **superpixels** or a coarse grid as the players (as in LIME).
- "Missing" features must be replaced by something: a **baseline** (blur, mean colour) or samples from a **background dataset**. This choice changes the answer.
- Aggregating SHAP values over many inputs gives a **global** view of which features matter overall.

Libraries: `shap`, `lime`, `captum` (PyTorch).

LIME vs SHAP

	LIME	**SHAP**
Idea	local linear surrogate	fair share of the prediction (Shapley)
Theory	heuristic; kernel chosen by the user	axiomatic; unique under fairness properties
Sums to prediction?	no	yes (efficiency)
Stability	can vary between runs	more stable; exact in special cases
Cost	moderate	high if model-agnostic; fast variants exist
Scope	local	local, and global when aggregated

- Both are **perturbation-based** and share a weakness: perturbed inputs can be **out-of-distribution**, and results depend on the **baseline / segmentation**.
- KernelSHAP shows they are relatives: SHAP $\approx$ LIME with a principled weighting.

Outline

1. Feature Visualization
2. Feature Attribution
   - Model-agnostic: LIME and SHAP
3. Concepts
4. Interpretable by Design: Concept Bottlenecks
5. Explaining Sequence Models
   - Token Attribution Basics
   - Explaining Transformers
6. Interpreting Vision-Language Models

Abstract Concepts Learned by Neural Network

- Imagine you build a ML model to switch traffic light that can detect fire-engines using cameras and give priority.
- If the model is trained using data from Melbourne, Can it be used in Canberra?



Fire Engine Melbourne

Abstract Concepts Learned by Neural Network

- Imagine you build a ML model to switch traffic light that can detect fire-engines using cameras and give priority.
- If the model is trained using data from Melbourne, Can it be used in Canberra?



Fire Engine Melbourne



Fire Engine Canberra

Abstract Concepts Learned by Neural Network

- Imagine you build a ML model to switch traffic light that can detect fire-engines using cameras and give priority.
- If the model is trained using data from Melbourne, Can it be used in Canberra?
- Does you fire engine detection model rely too heavily on Color?

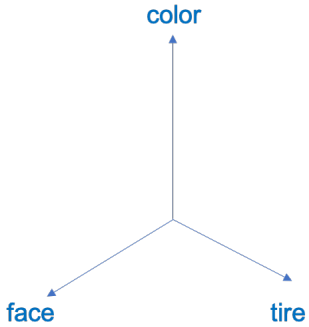


Fire Engine Melbourne

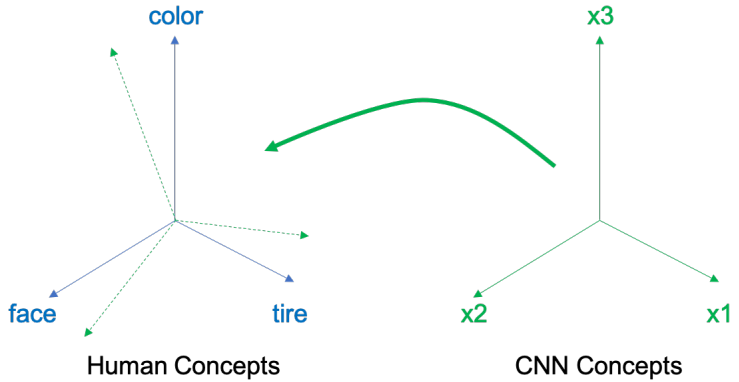


Fire Engine Canberra

Provide an interpretation of a neural net's internal state in terms of human-friendly concepts.



Provide an interpretation of a neural net's internal state in terms of human-friendly concepts.



Provide an interpretation of a neural net's internal state in terms of human-friendly concepts.

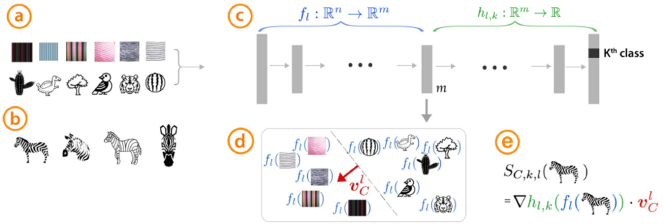


Image: Kim, B. et. al . Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV). ICML 2018.

[Step a] A concept is represented by a set of images:

- Some example image patches of the concept (e.g. stripes, red color).
- Some random image patches that does not belong to the concept.

Provide an interpretation of a neural net's internal state in terms of human-friendly concepts.

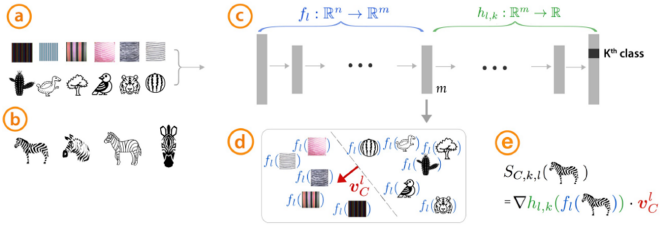


Image: Kim, B. et. al . Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV). ICML 2018.

[Step b] Select a set of images from the category we are interested in exploring (e.g. Zebra, Fire truck).

Provide an interpretation of a neural net's internal state in terms of human-friendly concepts.

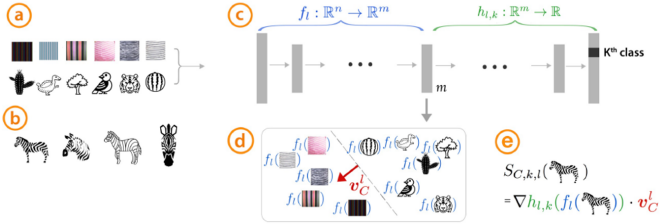


Image: Kim, B. et. al . Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV). ICML 2018.

[Step c] Forward propagate examples set (step a) through the CNN to get intermediate feature activation at a pre defined layer.

Provide an interpretation of a neural net's internal state in terms of human-friendly concepts.

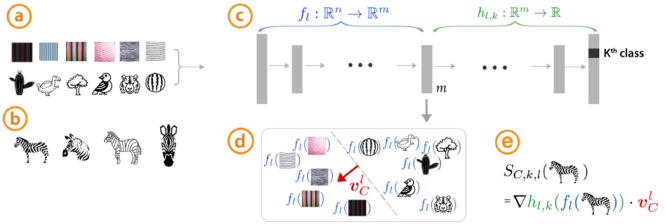


Image: Kim, B. et. al . Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV). ICML 2018.

[Step d] Do logistic regression on activations (from step c) to get the vector perpendicular to decision boundary v_C^l.

Provide an interpretation of a neural net's internal state in terms of human-friendly concepts.

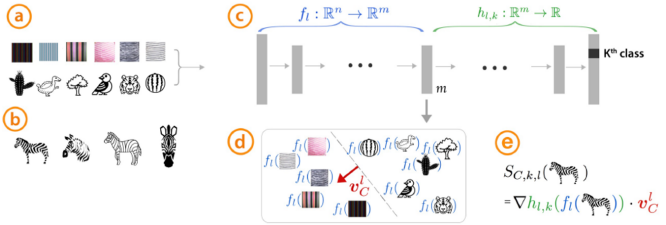


Image: Kim, B. et. al . Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV). ICML 2018.

[Step e] Compute the gradinets of class (e.g. Zebra, fire engine) with respect to hidden feature activation for for images from the class (step b).
Compute the similarity between that vector and v_C^l.
if the similarity is high the concept is important.

Outline

1. Feature Visualization
2. Feature Attribution
   - Model-agnostic: LIME and SHAP
3. Concepts
4. Interpretable by Design: Concept Bottlenecks
5. Explaining Sequence Models
   - Token Attribution Basics
   - Explaining Transformers
6. Interpreting Vision-Language Models

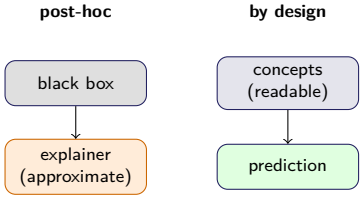
Post-hoc vs Interpretable by Design

Everything so far is **post-hoc**: train a black box, then try to explain it.

- Saliency, Grad-CAM, TCAV ... *approximate* what the model did.
- The explanation can be **unfaithful** – plausible-looking but not how the model actually decided.

Interpretable by design: build the model so its reasoning is *part of the architecture* and can be read directly.

- Preferred in **high-stakes** domains such as medicine, where a clinician must be able to check *why*.



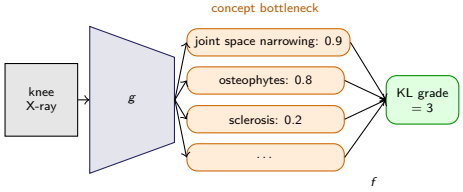
TCAV asks *"does the model use concept c?"* after training. A concept bottleneck *forces* the model to reason through concepts.

Split the network into two stages with a human-meaningful layer in between:

- g: input $x \to$ **concepts** $\hat{c} \in [0,1]^k$
  (e.g. "osteophytes present", "joint space narrowed")
- f: concepts $\hat{c} \to$ **label** $\hat{y}$
  (e.g. disease grade)

$$\hat{y} = f(g(x))$$

The prediction may depend on the image *only through* the concepts – that is the **bottleneck**. To explain a prediction, read the concept layer.



Training a CBM

Training needs **concept labels** c as well as class labels y. Three ways to train:

- **Independent**: train g on (x, c) and f on the *true* concepts (c, y) separately.
- **Sequential**: train g first, then train f on g's *predicted* concepts.
- **Joint**: train both together with a weighted loss

$$\mathcal{L} = \mathcal{L}_y(f(g(x)), y) + \lambda \sum_{j=1}^{k} \mathcal{L}_{c_j}(g_j(x), c_j).$$

The trade-off:

- Large λ: concepts are accurate and meaningful, but task accuracy may drop.
- Small λ: better task accuracy, but the "concepts" drift away from their clinical meaning.

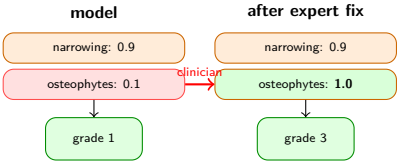
Koh et al. found CBMs can reach accuracy *comparable* to standard end-to-end models on their tasks – interpretability need not cost much.

Test-time Intervention

The key practical advantage: an **expert can correct the concepts** and the prediction updates.

1. The model predicts concepts and a diagnosis.
2. A radiologist spots a wrong concept (e.g. "no osteophytes" when they are clearly present).
3. Overwrite that concept value; f recomputes $\hat{y}$.

A black box offers nothing to edit – a CBM turns the model into a **human–AI collaborator**. Correcting only a few concepts can substantially improve accuracy.



Medicine already reasons in concepts – clinical guidelines list the findings that lead to a diagnosis. These map naturally onto a concept layer:

Task	Concepts (bottleneck)	Label
Knee osteoarthritis (X-ray)	joint space narrowing, osteophytes, sclerosis	Kellgren–Lawrence grade
Skin lesions (dermoscopy)	7-point checklist: pigment network, streaks, blue-whitish veil, ...	melanoma vs benign
Chest X-ray	radiological findings: opacity, effusion, cardiomegaly, ...	diagnosis / triage

- The explanation uses the **clinician's own vocabulary**, not pixel heatmaps.
- Errors become **diagnosable**: was a concept misdetected, or were the concepts combined wrongly?
- Supports audit and trust – important for regulatory approval of clinical AI.

Limitations and Extensions

Limitations

- **Annotation cost**: every training image needs concept labels – expensive expert time in medicine.
- **Incompleteness**: if the concept set misses important information, accuracy suffers.
- **Concept leakage**: soft concept scores can smuggle in extra, non-concept information, so the "explanation" is not the whole story.

Extensions

- **Label-free / post-hoc CBMs**: use a vision–language model (e.g. CLIP) to score concepts from text descriptions – no manual concept labels.
- **Concept embedding models**: richer concept representations that recover accuracy while keeping intervention.
- **Actively choosing** which concepts to annotate or query, to cut labelling cost.

Outline

1. Feature Visualization
2. Feature Attribution
   - Model-agnostic: LIME and SHAP
3. Concepts
4. Interpretable by Design: Concept Bottlenecks
5. Explaining Sequence Models
   - Token Attribution Basics
   - Explaining Transformers
6. Interpreting Vision-Language Models

What Does an Explanation Look Like for Text?

The vision question "which *pixels* mattered?" becomes "which *tokens* mattered?". The usual output is a **token heatmap**: a relevance score per token for a chosen prediction.



red = pushes towards *negative*, blue = pushes towards *positive*.

Two families of methods – the same as for images:

- **Perturbation-based**: remove or replace tokens and watch the output change.
- **Gradient-based**: use derivatives of the output with respect to the token embeddings.

Leave-one-out (occlusion for text)

- Delete (or mask) token i and measure the change in the predicted score:

$$R_i = f(x) - f(x_{\setminus i}).$$

- The text version of occlusion from the vision part.
- Simple, model-agnostic; misses interactions ("not" + "bad").

LIME / SHAP for text

- Exactly the methods from the vision part – only the *players* change: **words** instead of superpixels.
- Perturb by **removing or masking words**, query the model, then fit the local surrogate (LIME) or compute Shapley values (SHAP).
- Captures interactions better than leave-one-out, because many words are removed together.

Works on *any* model – even behind an API – but needs many forward passes.

Tokens are discrete, so take gradients with respect to their **embeddings** e_i.

- **Gradient $\times$ input**: $R_i = \sum_d \frac{\partial f}{\partial e_{i,d}} e_{i,d}$.
- **Integrated Gradients** Sundararajan et al. ICML 2017: accumulate gradients along a path from a *baseline* e' to the input:

$$R_i = (e_i - e'_i) \cdot \int_0^1 \frac{\partial f(e' + \alpha(e - e'))}{\partial e_i} \, d\alpha.$$

The scores sum to $f(e) - f(e')$ (**completeness**).

Choosing the baseline matters

- common choices: all-zero embeddings, or every token replaced by `[PAD]` / `[MASK]`;
- the explanation answers "relevant *compared to what*?" – a different baseline can give a different heatmap.

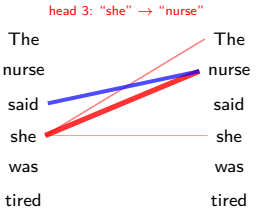
These work for RNNs, LSTMs and Transformers alike – they only need gradients.

Looking at Attention

Transformers compute attention weights $A = \mathrm{softmax}(QK^\top/\sqrt{d_k})$ in every head of every layer. A tempting explanation: *"the model attended to these tokens"*.

- Visualise one head as an $n \times n$ matrix, or as lines between tokens.
- Some heads show clear patterns: previous-token, syntactic (verb $\rightarrow$ subject), coreference.

But a model has *many* heads $\times$ layers, and attention in layer ℓ mixes *already-mixed* representations.



Is Attention an Explanation?

"Attention is not Explanation" Jain & Wallace

NAACL 2019

- Attention weights often correlate poorly with gradient- or perturbation-based importance.
- One can often find very *different* attention distributions that give the *same* prediction.

"Attention is not not Explanation"

Wiegreffe & Pinter EMNLP 2019

- Those alternative distributions are found adversarially; a model *trained* with them may behave differently.
- Attention can be *an* explanation – it depends on what you claim.

Practical take-away

- Attention shows *where information flows*, not *how much it mattered* to the output.
- Values V, residual connections and the feed-forward layers are ignored by raw attention.
- Treat raw attention as a **hint**; verify with gradients or perturbations.

One layer's attention says little about the *input* tokens: after layer 1 every position is already a mixture. **Rollout** tracks the flow back through all layers.

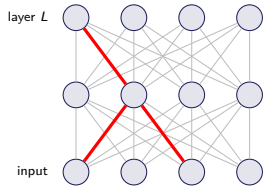
- Average heads, and add the identity to account for the **residual** connection:

$$\hat{A}^{(\ell)} = \frac{1}{2}(\bar{A}^{(\ell)} + I) \quad \text{(rows re-normalised)}.$$

- Multiply through the layers:

$$\tilde{A} = \hat{A}^{(L)}\,\hat{A}^{(L-1)} \ldots \hat{A}^{(1)}.$$

- Row `[CLS]` of $\tilde{A}$: how much each input token flowed into the classification token.



Follow the paths from each input token to the output.

Rollout is **class-agnostic**: it gives the same map whatever class you ask about. Weight attention by its **gradient** towards the target class – the Grad-CAM idea, applied to attention:

$$\bar{A}^{(\ell)} = \mathbb{E}_h\Big[(\nabla A_h^{(\ell)} \odot A_h^{(\ell)})^+\Big], \qquad R \leftarrow R + \bar{A}^{(\ell)} R,$$

starting from $R = I$ and updating layer by layer.

- $(\cdot)^+$ keeps only positive contributions; averaging over heads h weights heads by relevance.
- **Class-specific**: different maps for "cat" vs "dog".
- Also defined for encoder–decoder and **bi-modal** (text–image) attention.

Connection to earlier slides

CNN	Transformer
feature map	attention map
Grad-CAM weight	gradient of attention
CAM heatmap	token / patch relevance

Works for ViT too: relevance over image *patches* gives a heatmap.

Probing classifiers

- Freeze the model; train a small (usually linear) classifier on layer-ℓ representations to predict a property: part of speech, syntax depth, sentiment ...
- High probe accuracy $\Rightarrow$ the information is *linearly decodable* at that layer.
- Classic finding: lower layers encode more surface and syntactic features, higher layers more semantic ones.
- Caveat: decodable $\neq$ *used* by the model.

Logit lens

- In a language model, apply the final unembedding to an *intermediate* layer's hidden state.
- Read off "what the model would predict if it stopped here".
- Watch the prediction form layer by layer – e.g. the correct next word appearing only in later layers.

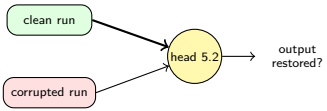
Both are *global* tools: they describe what the network represents, not why one output happened.

Goal: **reverse-engineer** the algorithm inside the network – not just *where* it looks, but *what computation* it performs.

- **Circuits**: small sub-networks of heads and neurons implementing a behaviour. Example: *induction heads* Olsson et al. 2022 copy patterns ("A B ... A $\rightarrow$ B"), a mechanism behind in-context learning.
- **Features via sparse autoencoders**: neurons are *polysemantic*; an SAE trained on activations recovers more interpretable directions ("DNA sequences", "legal text", ...).

Activation patching (causal tracing)

1. Run a *clean* input and a *corrupted* input.
2. Copy one component's activation from the clean run into the corrupted run.
3. If the correct output comes back, that component is **causally** important.



An intervention, not a correlation – the strongest evidence for faithfulness.

Choosing a Method for Transformers

Method	Needs weights?	Class-specific?	Cost	Main caveat
Raw attention	yes	no	1 pass	flow, not importance
Attention rollout	yes	no	1 pass	same map for every class
Leave-one-out / LIME / SHAP	**no**	yes	many passes	slow; misses interactions
Integrated Gradients	yes	yes	$\sim$20–300 passes	baseline choice
Gradient-weighted attention	yes	yes	1 fwd + 1 bwd	heuristic combination
Probing / logit lens	yes	–	cheap	decodable $\neq$ used
Activation patching	yes	yes	many passes	needs clean/corrupt pairs

- Local question ("why this prediction?") $\rightarrow$ IG, gradient-weighted attention, or LIME/SHAP.
- Global question ("what has the model learned?") $\rightarrow$ probing, logit lens, mechanistic tools.
- Always **sanity-check**: delete the top tokens – does the prediction actually change?

Outline

1. Feature Visualization
2. Feature Attribution
   - Model-agnostic: LIME and SHAP
3. Concepts
4. Interpretable by Design: Concept Bottlenecks
5. Explaining Sequence Models
   - Token Attribution Basics
   - Explaining Transformers
6. Interpreting Vision-Language Models

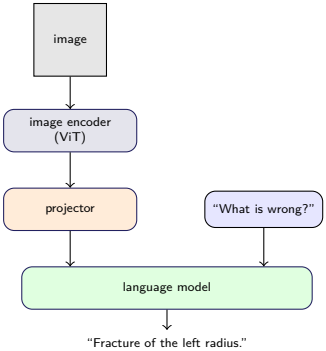
Vision-Language Models: What Do We Explain?

A typical VLM (e.g. LLaVA, Qwen-VL):

- an **image encoder** (often a ViT from CLIP) turns the image into patch tokens;
- a **projector** maps them into the language model's embedding space;
- a **language model** reads image tokens + text tokens and generates the answer.

Two questions we can ask:

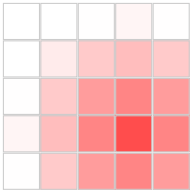
- **Grounding**: which image *regions* supported this word or answer?
- **Reasoning**: *why* did it give this answer?



Grounding: Where Did the Model Look?

Everything from this lecture transfers, now across modalities:

- **Attention from text to image tokens**: for an output word, which image patches did it attend to? Reshape to the patch grid → a heatmap. (Same caveats as raw attention.)
- **Gradient-weighted relevance** across the text–image attention (Chefer et al.) gives class- or word-specific maps.
- **CLIP similarity maps**: Grad-CAM on the image–text similarity score shows where "a fracture" appears in the image.



relevance of image patches for the word "fracture"

A generative VLM can **explain itself in words**:

> **Q**: Is there a fracture? Explain your answer.
> **A**: Yes. There is a lucent line across the distal radius with slight displacement, which indicates a fracture.

- Readable, in the domain's vocabulary, no extra tools.
- But is it the *real* reason? The explanation is *generated text* – produced by the same model, optimised to sound right.

Plausible $\neq$ faithful

- **Plausible**: convincing to a human.
- **Faithful**: reflects the computation that actually produced the output.

Language models can give fluent reasons that ignore the factor that truly drove the answer (e.g. a biasing cue in the prompt)

Turpin et al. NeurIPS 2023.

1. **Free to obtain – but not free of risk**
   - A text explanation costs one prompt, so it *feels* free.
   - It is not guaranteed to be **faithful**: it may describe evidence the model never used, or hallucinate findings.
   - Treat it as a **claim to be tested**, not ground truth – especially in medicine.

2. **What access do you have?**

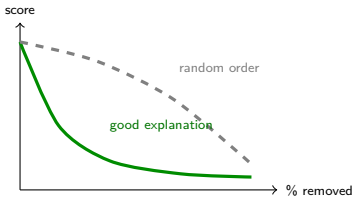
	open weights	closed API
self-explanation	✓	✓
perturbation / LIME	✓	✓
attention maps	✓	✗
gradients, IG	✓	✗
probing, patching	✓	✗

With a closed model you are limited to *black-box* methods and what it chooses to say.

Checking Faithfulness

Whatever the method, test whether the explanation *predicts the model's behaviour*:

- **Deletion test**: remove the top-ranked tokens or image regions – the prediction should drop quickly. Remove low-ranked ones – it should barely change.
- **Counterfactual consistency**: if the model says "because of the lucent line", edit out the line – does the answer change?
- **Randomisation check**: explanations should change when model weights are randomised; if not, they reflect the input, not the model.



A faithful explanation makes the score collapse fastest.

Summary

	Methods	Scope
Feature visualisation	filters, activations, gradient ascent	global
Feature attribution	occlusion, saliency, CAM, Grad-CAM; model-agnostic LIME, SHAP	local
Concepts: post-hoc	TCAV	global
Concepts: by design	concept bottleneck models, test-time intervention	both
Sequence models	LOO, LIME/SHAP, IG, rollout, gradient-weighted attention; probing, logit lens, patching	both
Vision–language	cross-modal relevance maps, self-explanations	local

- **Post-hoc** explanations approximate a black box; **by-design** models expose their reasoning.
- Explanations must be **faithful**, not just plausible – always test them.
- Your **access** (open weights vs API) decides which methods you can use.

Next week: Revision. **Lab**: Model Interpretation.